



Avaliação de Arquitetura

Mtech Cadastro de Clientes

Análise para evolução do sistema de apoio a vendedores
a uma plataforma CRM (estilo Pipedrive)

Repositório: [Du-Mt-26/cadastro-clientes](#)

Deploy: [mtech-clientes.vercel.app](#)

Framework: Next.js 16 + Prisma + PostgreSQL (Neon)



Data: 13 de Maio de 2026

Elaborado por: Z.ai

1. Sumário Executivo

Este documento apresenta uma avaliação abrangente da arquitetura do sistema "Cadastro de Clientes" da Mtech Geral, atualmente hospedado em mtech-clientes.vercel.app. O sistema foi desenvolvido como ferramenta de apoio aos vendedores de uma distribuidora de informática, com funcionalidades de gestão de carteiras de clientes, integração com Google Sheets, autenticação com 2FA e controle de acesso baseado em papéis (RBAC). A análise foi conduzida com o objetivo de identificar gargalos, riscos técnicos e oportunidades de melhoria que viabilizem a evolução do sistema para uma plataforma CRM completa, similar a soluções como Pipedrive ou HubSpot.

A arquitetura atual segue o padrão monolítico do Next.js com API Routes, Prisma ORM e PostgreSQL via Neon. Embora funcional para o escopo atual, o sistema apresenta problemas estruturais significativos que comprometem a escalabilidade, a manutenibilidade e a experiência de desenvolvimento. O componente principal (page.tsx) possui 1.335 linhas de código com 30+ estados React gerenciados manualmente, a camada de dados carrega todos os clientes em memória antes de filtrar, e não existe separação clara de responsabilidades entre camadas.

Visão Geral de Saúde da Arquitetura

Dimensão	Nota	Avaliação
Estrutura do Código	3/10	Componente monolítico de 1.335 linhas; sem separação de camadas
Camada de Dados	2/10	Carregamento completo em memória; cache ineficaz em serverless
Segurança	6/10	RBAC funcional, 2FA, mas sem CSRF, rate limiting ou validação de entrada
Escalabilidade	3/10	Filtros e paginação no lado do servidor mas após carregar tudo
Testabilidade	1/10	Zero testes; lógica de negócio acoplada à UI e rotas API
DX (Developer Experience)	4/10	TypeScript com erros ignorados; sem linting efetivo
Infraestrutura	7/10	Vercel + Neon é adequado; schema com indexes corretos
UI/UX	7/10	shadcn/ui profissional; dark mode; responsivo; bom para o escopo atual

2. Stack Tecnológico Atual

O sistema utiliza um stack moderno baseado em Next.js, o que é uma escolha sólida para aplicações web fullstack. No entanto, diversas decisões de implementação comprometem os benefícios que esse stack poderia oferecer. A tabela abaixo detalha cada tecnologia utilizada, sua versão e uma avaliação sobre a adequação da escolha e do uso que está sendo feito.

Tecnologia	Versão	Papel	Avaliação
Next.js	16.1.1	Framework fullstack (App Router)	Adequado, mas uso subótimo: página inteira é client-side
React	19.x	UI library	Adequado, mas sem uso de Server Components no page.tsx
Prisma	6.11.1	ORM (PostgreSQL/Neon)	Bom, mas queries deveriam filtrar no DB, não em memória
NextAuth.js	4.x	Autenticação	Funcional mas v4 está em modo legado; v5 disponível

Tecnologia	Versão	Papel	Avaliação
Tailwind CSS	4.x	Estilização	Excelente escolha, bem utilizado com shadcn/ui
shadcn/ui	-	Component library	Excelente, 40+ componentes instalados
Neon (PostgreSQL)	-	Banco de dados serverless	Adequado para Vercel; indexes bem definidos
xlsx	0.18.5	Import/Export Excel	Funcional mas pesado; considerar alternativas streaming
googleapis	171.x	Google Sheets API	Instalado mas não utilizado; sync usa CSV público
z-ai-web-dev-sdk	0.0.17	SDK de IA	Instalado mas sem uso aparente no código
framer-motion	12.x	Animações	Instalado mas uso mínimo no sistema
zod	4.x	Validação de schemas	Instalado mas NÃO utilizado nas API routes

[ALERTA] O arquivo `package.json` lista dependências não utilizadas (`googleapis`, `z-ai-web-dev-sdk`, `framer-motion`) que aumentam o `bundle size` e a superfície de ataque.

[CRÍTICO] `Zod` está instalado mas nenhuma API route utiliza validação de schema. Dados são aceitos diretamente do `request.body` sem validação.

[INFO] `NextAuth v4` está em modo de manutenção. A migração para `v5 (Auth.js)` deve ser planejada.

3. Análise Detalhada do Código-Fonte

3.1. Frontend: O Monólito de 1.335 Linhas

O arquivo `src/app/page.tsx` é o componente principal da aplicação e contém 1.335 linhas de código. Este único arquivo concentra toda a lógica de apresentação, gerenciamento de estado, manipulação de dados, filtros, paginação, exportação, modais de detalhes e formulários. O componente gerencia mais de 30 estados React com `useState`, incluindo estados para filtros, paginação, ordenação, modais, formulários, auditoria e sincronização com Google Sheets.

Problemas identificados:

- Violação do Princípio da Responsabilidade Única (SRP):** O componente `Home` é simultaneamente a view de listagem, o controller de filtros, o gerenciador de modais, o handler de exportação, o formulário de novo cliente e o visualizador de detalhes. Qualquer mudança em qualquer funcionalidade requer edição no mesmo arquivo gigante, aumentando exponencialmente o risco de regressões e conflitos em merges.
- Estado não gerenciado:** Com 30+ `useState`, o componente sofre de "prop drilling" severo. Estados como `saving`, `exporting`, `consulting`, `loading` são gerenciados individualmente em vez de utilizarem um reducer (`useReducer`) ou uma biblioteca de gerenciamento de estado como `Zustand` ou `Jotai`. A lógica de filtros interdependentes (cidade depende de UF, paginação reseta ao filtrar) está espalhada por múltiplos `useEffects` com dependências complexas.
- Filtragem client-side de "dias sem venda":** O filtro de dias sem venda é aplicado no cliente após receber todos os dados do servidor. Isso significa que se houver 10.000 clientes, todos são transmitidos pela rede para então serem filtrados no browser. A paginação funciona apenas para os dados já filtrados no servidor, criando inconsistência na contagem total.
- Fetch manual sem SWR/React Query:** As chamadas à API são feitas com `fetch` nativo dentro de `useEffect`, sem cache, deduplicação, revalidação automática ou tratamento de corrida (race

conditions). Se o usuário trocar de página rapidamente, respostas de requests antigos podem sobrescrever dados mais recentes.

5. Componente "use client" puro: page.tsx é inteiramente client-side, desperdiçando o maior benefício do Next.js App Router: Server Components. A página poderia renderizar dados iniciais no servidor, enviando HTML pronto ao cliente, com interatividade hidratada seletivamente.

3.2. Backend: API Routes com Lógica de Negócio Acoplada

As API routes do Next.js (localizadas em src/app/api/) funcionam como o backend do sistema. No total, são 15 rotas totalizando 1.820 linhas de código. Embora a separação em rotas distintas seja razoável, cada rota contém toda a lógica de autenticação, autorização, validação, acesso a dados e resposta HTTP inline, sem nenhuma camada de service ou repository.

Distribuição de linhas por rota:

Rota API	Linhas	Métodos	Observações
/api/clientes	420	GET, POST, PATCH	Rota principal; GET carrega tudo em memória
/api/vendedores	217	GET, POST, PATCH	CRUD de vendedores; duplica lógica de /api/users
/api/users	228	GET, POST, PATCH, DELETE	CRUD de usuários; admin-only
/api/clientes/bolsao	175	POST, PATCH	Lógica de carteira; N+1 queries em loop
/api/clientes/export	158	GET	Export XLSX; carrega tudo antes de filtrar
/api/clientes/receita	123	GET	Consulta ReceitaWS; sem rate limiting próprio
/api/sync	95	GET, POST, DELETE	Config Google Sheets
/api/vendedores/assign	86	PATCH	Atribuição cliente→vendedor
/api/auth/*	179	5 rotas	2FA setup/verify/disable + forgot-password
/api/sync/pull	39	POST	Pull do Google Sheets via CSV
/api/sync/push	27	POST	Stub: não implementado, retorna erro
/api/clientes/audit	22	GET	Busca logs; sem autenticação

[CRÍTICO] A rota GET /api/clientes carrega TODOS os registros do banco em memória (via getRecords()), filtra em JavaScript e depois pagina. Com 10K+ clientes, isso causará timeouts em serverless functions (limite de 10s no plano Hobby).

[CRÍTICO] A rota /api/clientes/audit NÃO verifica autenticação. Qualquer pessoa pode consultar o histórico de alterações de qualquer cliente.

[ALERTA] A rota /api/clientes/bolsao faz N+1 queries em loop (uma query UPDATE por cliente), quando poderia usar updateMany com uma cláusula WHERE adequada.

[ALERTA] /api/vendedores e /api/users possuem lógica duplicada de CRUD de usuários. Isso viola DRY e cria inconsistência.

3.3. Camada de Dados: Cache Ineficaz em Ambiente Serverless

O módulo `clientes-cache.ts` implementa um cache in-memory com TTL de 60 segundos. Embora a intenção seja legítima (reduzir chamadas ao banco), a implementação é fundamentalmente incompatível com o ambiente serverless da Vercel. Em serverless, cada request pode ser atendida por uma instância diferente da função, o que significa que o cache in-memory é recriado do zero a cada cold start. O cache só é efetivo durante a vida útil de uma mesma instância, que pode ser de segundos.

Fluxo problemático atual (GET /api/clientes):

1. `getRecords()` verifica se há cache em memória (variável global)
2. Se não houver, executa `db.cliente.findMany()` SEM filtros, carregando TODOS os registros
3. Converte todos os registros para `ClienteRecord` via `dbToRecord()`
4. Filtra os registros em JavaScript (por UF, cidade, situação, vendedor, carteira)
5. Ordena os registros em JavaScript
6. Calcula estatísticas iterando sobre todos os registros em JavaScript
7. Pagina os resultados em JavaScript

Esse fluxo é o oposto do que uma aplicação escalável deveria fazer. O banco de dados PostgreSQL via Neon suporta nativamente WHERE, ORDER BY, GROUP BY, COUNT, LIMIT e OFFSET com alta eficiência. Todo o processamento que é feito em JavaScript deveria ser delegado ao banco de dados, que é otimizado para essas operações. Com 5.000+ clientes, a rota GET pode facilmente exceder o tempo limite de serverless functions, especialmente em cold starts.

3.4. Schema Prisma: Modelo Adequado mas com Problemas

O schema Prisma define 4 modelos: `Cliente`, `User`, `SyncConfig` e `AuditLog`. A estrutura é razoável para o escopo atual, mas há problemas de design que dificultarão a evolução para CRM.

[ALERTA] O modelo `Cliente` possui 31 campos, quase todos strings com default vazio. Isso viola a normalização: dados como endereço deveriam ser uma entidade separada; telefones e emails deveriam ser listas, não campos numerados (`telefone1..4`, `email1..3`).

[ALERTA] O campo `"vendedoresQueAbordaram"` armazena IDs separados por vírgula em um campo Text. Isso impossibilita queries eficientes e viola a 1a Forma Normal. Deveria ser uma tabela de relacionamento N:N.

[INFO] O campo `"source"` (`xlsx|sheets|manual`) é mutuamente exclusivo, mas não há constraint. Um cliente pode ser importado e depois editado manualmente — qual é o source correto? Isso deveria ser um log, não um campo único.

[ALERTA] A coluna `"vendedor"` (string) e `"vendedorId"` (FK) são redundantes. O nome do vendedor é duplicado no cliente, criando risco de inconsistência. O nome deveria ser resolvido via JOIN.

[CRÍTICO] Não há modelo para `Contatos`, `Interações`, `Oportunidades` ou `Propostas` — entidades essenciais para evoluir a CRM. Adicionar essas entidades ao schema existente será complexo devido ao acoplamento atual.

4. Análise de Segurança

O sistema implementa uma base razoável de segurança com autenticação via NextAuth, criptografia `bcrypt` para senhas, suporte a 2FA via TOTP e middleware de proteção de rotas. No entanto, existem lacunas significativas que precisam ser endereçadas antes de qualquer

expansão do escopo.

Aspecto	Estado	Risco	Detalhe
Autenticação	Implementado	Baixo	NextAuth Credentials + JWT; bcrypt com salt 12
2FA (TOTP)	Implementado	Baixo	Google Authenticator compatível; setup/verify/disable
Autorização (RBAC)	Parcial	Médio	4 roles definidas mas verificação inconsistente nas rotas
Proteção CSRF	Ausente	Alto	Nenhum token CSRF em formulários; API aceita qualquer origin
Rate Limiting	Ausente	Alto	Sem limitação em endpoints de auth, consulta Receita ou export
Validação de entrada	Ausente	Crítico	Zod instalado mas NÃO utilizado; body aceito sem validação
Sanitização XSS	Parcial	Médio	React protege por padrão, mas observacoes é renderizado em textarea
Headers de segurança	Ausente	Médio	Sem CSP, X-Frame-Options, HSTS ou X-Content-Type-Options
Audit logging	Parcial	Médio	Log de edições mas rota de audit SEM autenticação
Sessão	Implementado	Baixo	JWT com maxAge de 8h; refresh automático via SessionProvider

[CRÍTICO] A ausência de validação de entrada (Zod está instalado mas não é usado) significa que qualquer campo pode receber dados malformados, inclusive injeção de dados em campos que não deveriam ser aceitos no PATCH.

[CRÍTICO] Sem rate limiting, um atacante pode realizar brute-force nas senhas ou sobrecarregar o endpoint de consulta à Receita Federal (que já tem rate limit próprio de 3 req/min).

[ALERTA] O next.config.ts tem ignoreBuildErrors: true, o que permite que erros TypeScript passem despercebidos no build, incluindo erros de tipo que poderiam indicar bugs de segurança.

5. Escalabilidade e Performance

O sistema atual funcionará de forma aceitável com poucas centenas de clientes e poucos usuários concorrentes. No entanto, o modelo de carregamento completo em memória cria um gargalo que se agrava linearmente com o volume de dados. A análise abaixo detalha os principais pontos de atenção e suas implicações práticas.

5.1. Gargalo Principal: Carregamento Full-Table

A função `getRecords()` em `clientes-cache.ts` executa `db.cliente.findMany()` sem nenhum filtro, carregando todos os registros do banco. Com o schema atual, cada registro `Cliente` possui 31 campos de string. Estimando uma média de 200 bytes por registro, uma base de 10.000 clientes geraria aproximadamente 2 MB de dados transferidos do banco por request. Com 50.000 clientes, isso sobe para 10 MB — insustentável para uma serverless function com timeout de 10 segundos.

A solução é straightforward: mover filtros, paginação, ordenação e agregações para queries Prisma com `WHERE`, `ORDER BY`, `LIMIT/OFFSET` e `GROUP BY`. O PostgreSQL é extremamente eficiente nessas operações, especialmente com os indexes já definidos no schema. Essa mudança sozinha reduziria o tempo de resposta de $O(n)$ para $O(\log n)$ na maioria das queries.

5.2. Cache In-Memory em Serverless

O cache implementado (TTL de 60s, variável global) é ineficaz no ambiente Vercel serverless. Cold starts recriam a instância, invalidando o cache. Warm starts podem compartilhar o cache dentro da mesma função, mas a Vercel não garante persistência de instâncias. A solução ideal para serverless é cache distribuído (Redis via Upstash ou Vercel KV) ou, preferencialmente, queries de banco eficientes que tornem o cache desnecessário para a maioria das operações.

5.3. N+1 Queries no Bolsão

A rota POST /api/clientes/bolsao faz um loop sobre todos os clientes elegíveis e executa uma query UPDATE individual para cada um. Com milhares de clientes, isso gera centenas de round-trips ao banco. O Prisma suporta updateMany com cláusulas WHERE compostas, que executaria a mesma operação em uma única query. Da mesma forma, a verificação de "carteira friia" faz um loop sobre clientes do bolsão com lógica de verificação em JavaScript que deveria ser uma query SQL.

6. Roadmap de Evolução para CRM

A transformação do sistema atual em uma plataforma CRM completa requer mudanças arquiteturais fundamentais. O roadmap abaixo propõe 4 fases, priorizando primeiramente a estabilização da base técnica (refatoração e correção de problemas críticos) e depois a adição incremental de funcionalidades de CRM. Cada fase é projetada para entregar valor independente, sem bloquear as fases subsequentes.

Fase 1 — Fundação (2-3 semanas)

Esta fase foca na correção de problemas críticos e na criação de uma base arquitetural sólida. Sem essa fundação, qualquer funcionalidade nova será construída sobre areia, acumulando dívida técnica que tornará mudanças futuras cada vez mais caras e arriscadas.

Tarefa	Prioridade	Impacto
Mover filtros/paginação/agregação para queries Prisma	Crítica	Performance 10x-100x; viabiliza crescimento
Adicionar validação Zod em todas as API routes	Crítica	Elimina risco de injeção e dados corrompidos
Autenticar rota /api/clientes/audit	Crítica	Correção de vulnerabilidade de exposição de dados
Adicionar rate limiting (Upstash/Vercel KV)	Alta	Proteção contra brute-force e abuso de API
Adicionar CSRF tokens e headers de segurança	Alta	Proteção contra ataques cross-site
Remover dependências não utilizadas	Média	Redução do bundle size e superfície de ataque
Ativar type checking (remover ignoreBuildErrors)	Média	Deteção precoce de bugs

Fase 2 — Refatoração Frontend (2-3 semanas)

Com a base de dados e segurança estabilizadas, a segunda fase foca na reorganização do frontend. O objetivo é transformar o componente monolítico de 1.335 linhas em uma arquitetura modular e manutenível, preparando o terreno para a adição de funcionalidades de CRM sem que o código se torne ingovernável.

Tarefa	Prioridade	Impacto
Dividir page.tsx em 8-10 componentes menores	Crítica	Manutenibilidade;□□□□ sem conflitos
Adicionar SWR ou TanStack Query para data fetching	Alta	Cache, deduplicação, revalidação automática
Migrar filtros para URL params server-side	Alta	URLs compartilháveis; SSR com dados filtrados
Adicionar useReducer ou Zustand para estado complexo	Média	Estado previsível; elimina useEffect cascata
Converter componentes de lista para Server Components	Média	Menos JavaScript no cliente; melhor SEO
Criar camada de services (clientes.service.ts, etc.)	Média	Separação de concerns; reutilização de lógica

Fase 3 — Entidades CRM (3-4 semanas)

Com a arquitetura refatorada, a terceira fase adiciona as entidades centrais de um CRM: contatos, interações, oportunidades e pipeline de vendas. Essas entidades permitem que os vendedores registrem atividades, acompanhem negociações e gestores tenham visibilidade do funil de vendas.

Novos modelos Prisma propostos:

- Contato — pessoas dentro de um cliente (substitui pessoaContato string)
- Interacao — registro de contato (ligação, email, visita, reunião)
- Oportunidade — negociação em andamento com valor, etapa e probabilidade
- Pipeline — definição de etapas do funil de vendas (configurável)
- Atividade — tarefas/agendamentos vinculados a oportunidades ou clientes
- Nota — observações estruturadas (substitui o campo observacoes string)

A adição desses modelos exigirá também a normalização do modelo Cliente: os campos telefone1-4 e email1-3 devem ser migrados para uma tabela de Contato com tipo (telefone_celular, telefone_fixo, email, etc.), e o campo vendedoresQueAbordaram deve virar uma tabela N:N entre Cliente e User. Essa migração é delicada e deve ser feita com migrações incrementais do Prisma, mantendo compatibilidade com a UI existente durante a transição.

Fase 4 — Features Avançadas (4-6 semanas)

A quarta fase adiciona funcionalidades que diferenciam um CRM básico de uma ferramenta realmente útil para a equipe comercial. Cada feature pode ser desenvolvida e entregue independentemente, permitindo que a equipe priorize o que traz mais valor imediato.

Feature	Descrição	Dependência
Pipeline Kanban	Visualização drag-and-drop do funil de vendas por etapa	Modelo Oportunidade + Pipeline
Dashboard de Métricas	Conversão por etapa, tempo médio de fechamento, tempo médio de resposta	Dados de Oportunidade acumulados
Automações	Regras: "se sem interação há 7 dias, notificar gestor"	Modelo Interacao + sistema de eventos
Integração Email	Sincronizar emails do vendedor com o CRM	Modelo Interacao + OAuth Gmail
WhatsApp Business API	Registro de conversas WhatsApp como interações	Modelo Interacao + Meta API
Relatórios Avançados	Relatórios de performance por vendedor, região, período	Dados acumulados de todas as entidades

Feature	Descrição	Dependência
Importação/Exportação	Import via CSV/XLSX com mapeamento de colunas	Refatoração da importação atual
Notificações	Push e email para lembretes, atribuições e alertas	Sistema de atividades + serviço de email

7. Arquitetura Alvo Proposta

A arquitetura proposta mantém o stack atual (Next.js + Prisma + Neon + Vercel) mas reorganiza as camadas de forma clara e escalável. O princípio central é a separação de responsabilidades: cada camada tem um propósito bem definido e não vaza abstrações para as camadas adjacentes.

7.1. Estrutura de Diretórios Proposta

A nova estrutura organiza o código em módulos de domínio (feature-based), onde cada funcionalidade agrupa seus próprios componentes, hooks, serviços e tipos. Isso contrasta com a organização atual que separa por tipo técnico (components/, lib/, api/) mas mistura domínios dentro de cada pasta.

```
src/
app/
(auth)/ # Route group: login, 2FA
(dashboard)/ # Route group: páginas autenticadas
clientes/ # Página de listagem de clientes
pipeline/ # Kanban de oportunidades
relatorios/ # Dashboard de métricas
api/
clientes/ # API routes de clientes
interacoes/ # API routes de interações
oportunidades/ # API routes de oportunidades
modules/
clientes/
components/ # ClientList, ClientDetail, ClientForm...
hooks/ # useClientes, useClienteFilters...
services/ # clientes.service.ts (lógica de negócio)
types/ # Tipos específicos do domínio
pipeline/
components/ # KanbanBoard, DealCard, StageColumn...
hooks/ # usePipeline, useDeal...
services/ # pipeline.service.ts
lib/
db/ # Prisma client + helpers
auth/ # NextAuth config + middleware + guards
validators/ # Schemas Zod compartilhados
utils/ # Helpers genéricos
```

7.2. Padrões Arquiteturais Recomendados

Service Layer Pattern: Toda lógica de negócio (filtros complexos, cálculos de bolsão, sincronização com sheets) deve ser extraída das API routes para services (ex: ClientesService, PipelineService). As rotas API ficam responsáveis apenas por: receber request, chamar o service apropriado e retornar a resposta. Isso permite testar a lógica de negócio isoladamente, reutilizá-la em diferentes contexts (API, server actions, cron jobs) e manter as rotas enxutas.

Repository Pattern (opcional): Se a complexidade das queries Prisma crescer, um repository pode abstrair o acesso a dados, permitindo trocar a implementação (ex: de Prisma para queries SQL brutas com `$queryRaw` para agregações complexas) sem alterar a camada de service. Para o escopo atual, services que usam Prisma diretamente são suficientes.

Server Actions + Server Components: O Next.js 16 suporta Server Actions, que permitem que o cliente invoque funções server-side diretamente sem criar API routes. Para operações simples (salvar edição, criar cliente), Server Actions eliminam a necessidade de rotas API dedicadas. Para operações que precisam de URLs públicas (webhooks, exports), mantêm-se API routes.

Optimistic UI com SWR/TanStack Query: Para operações de escrita (PATCH, POST), utilizar atualização otimista: atualizar o cache local imediatamente e revalidar em background. Isso dá a sensação de resposta instantânea ao usuário, essencial em um CRM onde vendedores precisam registrar interações rapidamente entre ligações.

8. Considerações Finais

O sistema Mtech Cadastro de Clientes demonstra um bom domínio do stack Next.js/Prisma/shadcn e entrega valor real para a equipe comercial. A interface é profissional, com dark mode, filtros ricos, edição inline, exportação e integração com Google Sheets. A funcionalidade de carteiras (Revendas, Corporativo, Bolsão, Fria) é um diferencial importante que reflete o processo comercial real da distribuidora.

No entanto, a arquitetura atual tem limitações estruturais que precisam ser endereçadas antes de qualquer expansão significativa. Os três problemas mais urgentes são: (1) o carregamento full-table em memória que não escala; (2) a ausência de validação de entrada que cria vulnerabilidades; e (3) o componente monolítico de 1.335 linhas que torna qualquer mudança arriscada e lenta. Esses problemas não impedem o funcionamento atual, mas se tornarão bloqueadores à medida que o volume de dados e a complexidade funcional aumentem.

A evolução para CRM é viável com o stack atual, desde que feita de forma incremental. O roadmap proposto em 4 fases permite que a equipe entregue valor a cada etapa, priorizando primeiro a estabilização da base técnica e depois a adição de funcionalidades. A chave do sucesso é não tentar construir o CRM completo de uma vez, mas sim refatorar gradualmente, mantendo o sistema sempre funcional e em produção.